

Batch: A-4

Roll No.: 16010122151

IA-2 Flip Classroom

IA2 – Flip Classroom

Marks: 20

1. In a churn prediction model, the confusion matrix for a test set of 500 customers is as follows: True Positive (TP) = 120, False Positive (FP) = 50, True Negative (TN) = 200, False Negative (FN) = 130. Calculate the accuracy, precision, recall, and F1-score of the model. (5 marks)

Q1.

	Predicted Positive	Predicted Negative
Actual Positive	120 (TP)	130 (FN)
Actual Negative	50 (FP)	200 (TN)

Given:

- True Positive (TP): 120 (Model correctly predicted churn)
- False Positive (FP): 50 (Model incorrectly predicted churn)
- True Negative (TN): 200 (Model correctly predicted no churn)
- False Negative (FN): 130 (Model incorrectly predicted no churn)
- Total: $TP + FP + TN + FN = 120 + 50 + 200 + 130 = 500$

a) Accuracy: • Formula: $\text{Accuracy} = \frac{TP + TN}{\text{Total}}$

• Calculation: $\frac{120 + 200}{500} = \frac{320}{500} = 0.64$

• Result: the model's accuracy is 64%

b) Precision: • Formula: $\text{Precision} = \frac{TP}{TP + FP}$

Teacher's Signature _____

- Calculation: $\frac{120}{120+50} = \frac{120}{170} \approx 0.706$
- Result: The model's precision is 70.6%.

c) Recall (Sensitivity):

• Formula: $\text{Recall} = \frac{TP}{TP+FN}$

• Calculation: $\frac{120}{120+130} = \frac{120}{250} = \underline{0.48}$

- Result: The model's recall is 48%.

d) F1-Score :

• Formula: $\text{F1-Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$

• Calculation: $2 \times \frac{0.706 \times 0.48}{0.706 + 0.48} = 2 \times \frac{0.3389}{1.186} = \underline{0.571}$

- Result: The model's score is 57.1%.

Analysis: The model is moderately precise (70.6%), meaning that when ~~a custom~~ it predicts a customer will churn, it's correct about 70% of the time. However, it suffers from low recall (48%), meaning it fails to identify more than half of the customers who actually did churn.

2. Implement a gradient descent algorithm for minimizing a differentiable convex function. Then, analyze how varying the learning rate affects convergence to the global minimum. Provide a mathematical justification using gradient magnitude and curvature. (5 marks)

2. a) Gradient Descent Algorithm Implementation:

1. Initialize Parameters: Start with initial values for the model's parameters (e.g. $m=0$ and $c=0$).
2. Compute Gradients: Calculate the partial derivative of the loss function w.r.t each parameter.
 - Gradient of Loss w.r.t m : $\frac{\partial \text{MSE}}{\partial m}$
 - Gradient of Loss w.r.t c : $\frac{\partial \text{MSE}}{\partial c}$
3. Update Parameters:
 - Update Rule:
 - $m_{\text{new}} = m_{\text{old}} - \alpha \times \frac{\partial \text{MSE}}{\partial m}$
 - $c_{\text{new}} = c_{\text{old}} - \alpha \times \frac{\partial \text{MSE}}{\partial c}$
4. Iterate: Repeat steps 2 & 3 until the loss converges to a minimum (i.e. the gradients are close to zero) or a maximum no. of iterations is reached.

b) Analysis of Learning Rate (α) on Convergence:

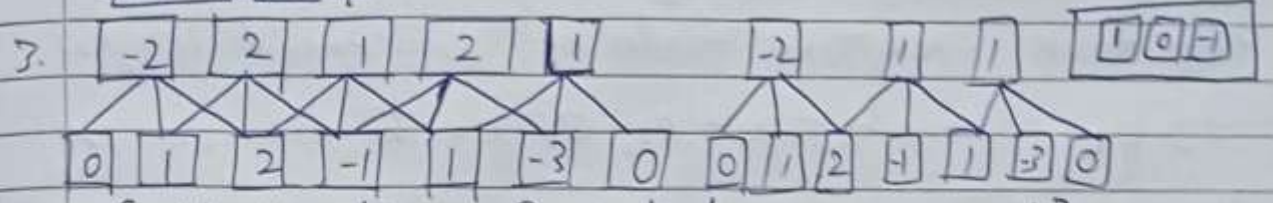
- If the learning rate is too low: The model will take very small steps at each iteration.

- If the learning rate is too high: The model will take very large steps. This can cause the algorithm to "overshoot" the minimum, leading the loss to oscillate, fail to converge, or even diverge (increase indefinitely).
- If the learning rate is just right: The model converges to the global minimum (for a convex function) efficiently and in a reasonable number of iterations.
- c) Mathematical Justification (Gradient Magnitude & Curvature): The parameter update is given by $w_{\text{new}} = w_{\text{old}} - \alpha \nabla L(w)$.
 - Gradient Magnitude ($\|\nabla L(w)\|$): This determines the size of the gradient. When the slope is steep (far from the minimum), the gradient magnitude is large, resulting in a large step. As the model approaches the minimum, the slope flattens, the gradient magnitude becomes small, and steps auto-shorten.
 - Curvature:
 - In a region of (high curvature), the gradient changes rapidly.
 - In a region of low curvature, the gradient

is small.

For convergence, the learning rate must be inversely proportional to the curvature. A high α can cause divergence because the step $\alpha \nabla L(w)$ repeatedly overshoots the minimum, sending the parameters farther away with each step.

3. CNNs automatically extract spatial hierarchies of features. Analyze how filter size, stride, and padding influence feature extraction and receptive field size. Discuss how these parameters affect overfitting and model generalization when applied to high-resolution images. (5 marks)

3. 

[Illustration of spatial arrangement]

a) Influence of Filter size, stride and padding:

- Filter Size (f): This defines the dimensions of the kernel (e.g. 3×3) that slides over the input.
- Feature Extraction: The filter's weights are what learn to detect specific patterns.
- Receptive Field: The filter size directly determines the the local ~~repetitive~~ receptive field of a single neuron in that layer.

→ A small filter captures fine-grained, local features like edges, corners and textures.
 A large filter captures larger, more complex shapes or patterns.

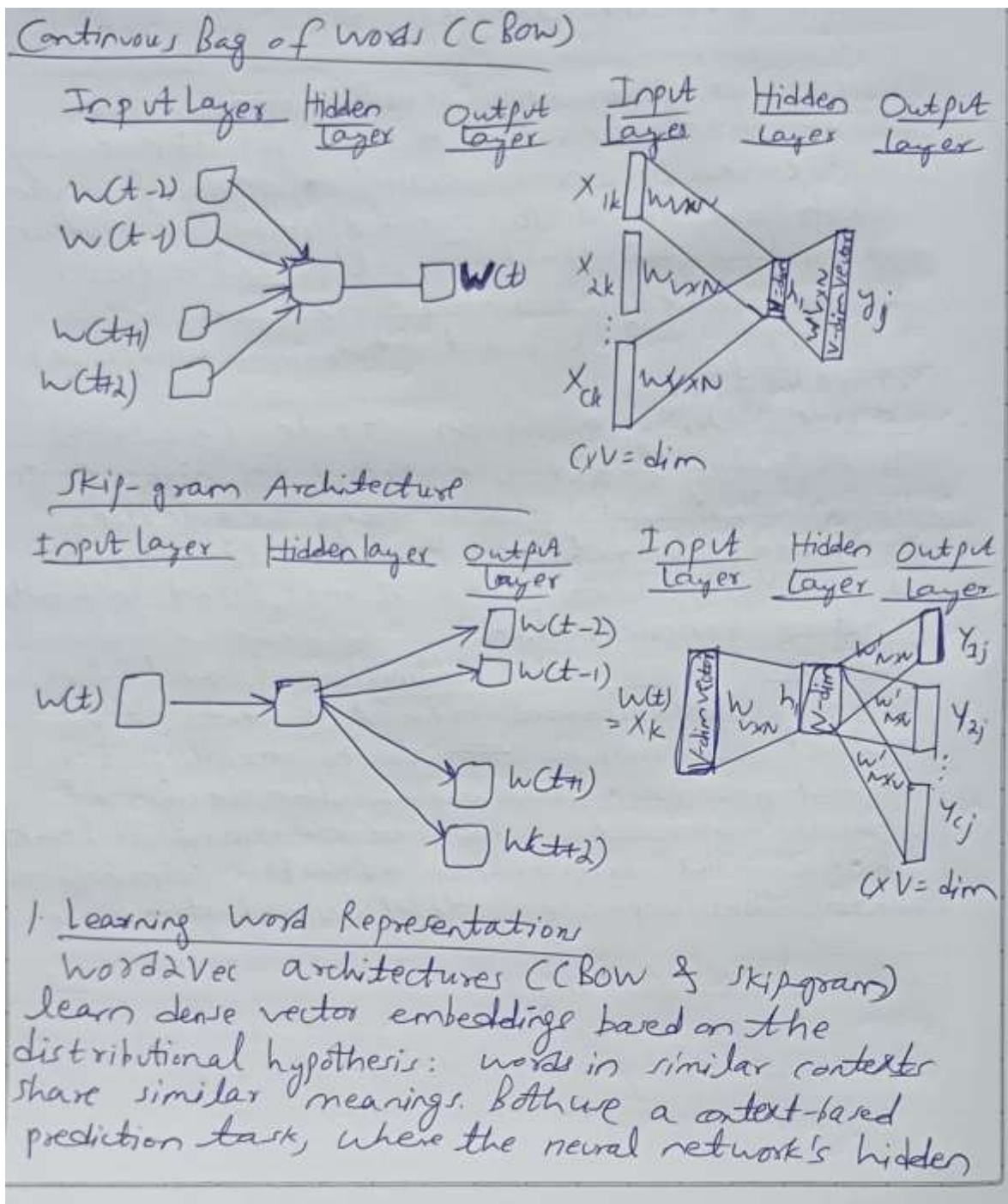
Teacher's Signature _____

- Stride(s): This is the step size, or the number of pixels, the filter moves at a time.
- Feature Extraction: A stride of 1 ($s=1$) is dense sampling every possible location. A stride greater than 1 ($s>1$) performs sparse sampling. This acts as a form of downsampling, reducing the spatial dimensions (width & height) of the output feature map.
- Receptive Field: A larger stride causes the effective receptive field of subsequent layers to grow much faster, as each neuron in the next layer will correspond to a larger area of the original input.
- Padding(p): This is the process of adding pixels (usually zeros) to the border of the input volume before convolution.
 - Feature Extraction: It ensures that the filter can be applied to the pixels at the very edge of the image.
 - Receptive Field: Padding does not directly alter the local receptive field. Its main role is to preserve spatial resolution, which allows for the construction of deeper networks.

b) Effect on Overfitting and Generalization (High-Resolution images) :

- Filter Size: For high-resolution images using large filters (e.g. 7×7 or 11×11) in the early layers results in a massive no. of params, which can lead to overfitting. The modern approach is to stack small 3×3 filters. It has fewer parameters and an extra non-linear (ReLU) activation, which improves generalization.
- Stride: Using a larger stride (e.g. $s=2$) or a pooling layer is essential for high-resolution images. This acts as a strong regularizer, reducing overfitting. However, too large a stride can discard important information, hurting generalization.
- Padding: "Same" padding is crucial for high-resolution images as it prevents the spatial dimensions from shrinking too quickly. This allows for the design of very deep networks, which are necessary to learn the complex hierarchy of features in high-res data, thus improving model generalization.

4. Analyze how the Continuous Bag of Words (CBOW) and Skip-gram architectures in Word2Vec learn word representations using context-based prediction. Discuss the differences in their training objectives and explain how these differences influence the quality and type of word embeddings produced. (5 marks)



layer weights become the final word embeddings.

2. Architectural & Training Objective Differences

CBOW: Continuous Bag of words learns by predicting the target word from its surrounding context words (a many-to-one task).

• Example:- { "the", "cat", "on", "the" } $\rightarrow$ "sat"

• Process:- It averages the context word embeddings into a single "context vector" to make this prediction.

• Objective:- TO maximise the probability of the target word given the context, $P(w_t | c)$.

Skip-gram: Does the inverse, predicting context words from a single target word (a one-to-many task).

• Example:- "sat" $\rightarrow$ { "The", "cat", "on", "the" }.

• Process:- It uses the single target embedding to make multiple predictions, creating several training examples (e.g., (sat, The), (sat, cat)).

• Objective:- Maximise probability of the context given the target word, $P(c | w_t)$.

2. Influence on Embedding Quality & Type:

Difference in training objectives:

Feature	CBOw	Skip-gram
Training Speed	<u>Faster</u> . By averaging the context, it has only one prediction task per target word.	<u>Slower</u> . It generates multiple prediction tasks (training samples) for each target word.
Frequent words	<u>Handled well</u> . Its stable and efficient for frequent words.	Good, but the multiple predictions for frequent words (like "the") are computationally expensive.
Rare words	<u>Struggles</u> . The meaning of a rare word is "averaged" out or diluted by its most common context words.	<u>Excellent</u> . By being forced to predict the context from the rare word, the model must learn a high-quality, fine-grained rep ⁿ for it.
Embedding Type	Tends to be slightly better at learning syntactic relationships, e.g. cat's "eat" vs "eats".	Tends to be better at learning semantic relationships & analogies (e.g. "king - man + woman = queen").

Summary of Influence:

• CBOw is faster and performs well for frequent words because its objective is simpler & more efficient. This "smoothing" effect makes it good for general syntactic understanding.

• Skip-gram is slower but produces higher-quality embeddings for rare words & complex semantic relationships. Its "one-to-many" objective is a more difficult task, forcing the model to learn more detailed & robust representations for each individual word.